

#1

JavaScript Closures

QUESTION

What is a closure in JavaScript? Provide an example.

APPROACH

Explain the concept, then illustrate with a practical example

ANSWER

A closure is a feature where an inner function has access to the outer (enclosing) function's variables.

Example:

```
javascript Copy code  
  
function outerFunction(outerVariable) {  
  return function innerFunction(innerVariable) {  
    console.log('Outer Variable: ' + outerVariable);  
    console.log('Inner Variable: ' + innerVariable);  
  }  
}  
  
const newFunction = outerFunction('outside');  
newFunction('inside');
```

#2

Responsive Design

QUESTION

How would you implement a design that needs to be responsive across various devices?

APPROACH

Discuss the use of media queries, flexible grid layouts, and relative units.

ANSWER

Utilize CSS media queries to apply different styles based on device characteristics like width, height, or orientation. Employ a fluid grid layout and use relative units (em, rem, %) instead of pixels to ensure elements scale proportionally.

#3

JavaScript Promises

QUESTION

Explain the difference between `.then()` and `async/await` in handling asynchronous operations in JavaScript.

APPROACH

Compare both, highlighting syntax and usability differences.

ANSWER

`.then()` is used with promises for asynchronous operations, chaining multiple calls for sequential execution. `async/await` makes asynchronous code look synchronous and is syntactic sugar over Promises, improving readability and error handling.

#4

React Component Lifecycle

QUESTION

Describe the lifecycle of a React component and how you would use it to fetch data.

APPROACH

Detail the lifecycle phases and appropriate methods for data fetching.

ANSWER

React components have several lifecycle methods: `constructor()`, `render()`, `componentDidMount()`, `shouldComponentUpdate()`, `componentDidUpdate()`, and `componentWillUnmount()`. To fetch data, use `componentDidMount()` for class components or `useEffect()` hook in functional components to perform side effects, including data fetching after the initial render.

#5

Cross-Origin Resource Sharing (CORS)

QUESTION

What is CORS and how do you handle it in web applications?

APPROACH

Explain CORS concept and solutions for common issues.

ANSWER

CORS is a security feature that restricts web applications from making requests to a domain different from the one which served the web page. To handle it, configure the server to include CORS headers like `Access-Control-Allow-Origin` in the response, specifying which domains are allowed to access the resources.

#6

Front-End Performance Optimization

QUESTION

What strategies would you employ to optimize the performance of a web application?

APPROACH

List practical techniques for improving front-end performance.

ANSWER

Optimization strategies include minimizing and compressing assets (CSS, JavaScript, images), implementing lazy loading for images and components, using CDN for static assets, optimizing CSS selectors, and leveraging browser caching.

#7

Single Page Application (SPA) SEO Challenges

QUESTION

What are the SEO challenges with SPAs and how can they be addressed?

APPROACH

Identify key SEO issues with SPAs and solutions.

ANSWER

SPAs often struggle with SEO as content is dynamically loaded, making it hard for search engine crawlers to index. Solutions include server-side rendering (SSR), using the History API to update URLs for different views, and leveraging pre-rendering services or static site generators.

#8

Event Delegation in JavaScript

QUESTION

Explain event delegation and its advantages.

APPROACH

Describe the concept and its practical uses.

ANSWER

Event delegation is a technique where instead of adding an event listener to each similar child element, you add a single event listener to a parent element. It leverages the event bubbling phase to catch events from child elements. Advantages include reduced memory usage (fewer event listeners) and dynamically handling events from elements added after the initial page load.

Beginner Level

1. What is JavaScript?

JavaScript is a high-level, interpreted programming language primarily used for creating interactive web pages.

2. What are the different data types in JavaScript?

JavaScript supports various data types including strings, numbers, booleans, null, undefined, objects, and symbols (added in ES6).

3. How do you declare variables in JavaScript?

Variables in JavaScript can be declared using the `var`, `let`, or `const` keywords.

4. What is the difference between `let`, `const`, and `var`?

`var` has function scope, `let` has block scope, and `const` is used to declare constants whose value cannot be reassigned.

5. What is the use of `==` and `===` operators in JavaScript?

`==` is used for loose equality comparison, whereas `===` is used for strict equality comparison, which checks both value and type.

6. How do you comment in JavaScript?

Single-line comments are denoted by `//`, and multi-line comments are enclosed between `/*` and `*/`.

7. What are JavaScript primitive data types?

Primitive data types in JavaScript include strings, numbers, booleans, null, undefined, and symbols.

8. What is the `typeof` operator used for?

The `typeof` operator is used to determine the type of a variable or an expression.

9. Explain hoisting in JavaScript.